

# Static Analysis Feedback in LLM Repair Loops: A Controlled Study of mypy Augmentation

Anonymous Authors<sup>1</sup>

## Abstract

LLM repair loops improve code generation quality by feeding execution output back to the model for iterative correction — but whether adding static analysis feedback provides meaningful improvement over execution-only feedback has not been tested in a controlled experiment. We conduct a five-sub-hypothesis study on HumanEval+ (164 problems, GPT-4o-mini, k=5 repair rounds) to isolate the marginal contribution of mypy type checking over execution-only feedback. Our primary finding: execution+mypy repair achieves 87.20% pass@1 versus 78.25% for execution-only — a consistent +8.94 percentage point improvement across three independent seeds (seed std:  $A=\pm 1.56pp$ ,  $B=\pm 0.49pp$ ; delta range: 6.7pp to 11.0pp). This improvement is not type-specific: execution-only repair achieves identical 90.9% repair rates on type-error problems (differential = 0.000), consistent with mypy functioning as a general diagnostic context enricher — a feedback channel that improves repair quality broadly rather than for a specific error type — though this mechanism interpretation has not been confirmed via ablation (see Section 6, L2). We further show that HumanEval+ and MBPP+ have fundamentally different mypy error profiles (70% vs 0% in failing solutions), predicting where static analysis feedback will and will not activate. A Z3 formal verification extension achieves 91.1% spec validity rate on arithmetic HumanEval+ problems but provides no additional pass@1 improvement at the current counterexample activation rate

(16.1%). These results offer controlled evidence that one structured feedback channel — a single mypy subprocess call — substantially improves LLM repair quality on HumanEval+, while establishing that the intuitive type-specificity mechanism is not the active driver.

## 1. Introduction

Large language models can repair their own code when given execution feedback: run the generated solution, capture the output or exception, and feed it back to the model for correction. This repair loop paradigm, exemplified by Self-Debug (Chen et al., 2024) and Reflexion (Shinn et al., 2023), has become a standard technique for improving pass@k on benchmarks like HumanEval and MBPP. The dominant feedback signal is execution output — a binary pass/fail plus stderr. The intuition is that more specific feedback should produce better repairs: if the model knows not just that it failed but why (a type error at line 5 with expected type `int`, received `str`), it should correct the error more reliably. Static analysis tools like mypy generate exactly this kind of structured, type-level diagnostic. Yet no controlled study has measured whether adding mypy to a repair loop produces measurable improvement over execution-only feedback.

The gap matters for a practical reason: execution feedback is cheap but coarse. mypy feedback is almost equally cheap (a subprocess call, approximately 30ms) but structured — it provides error category, file location, and error message. If this structure translates into better repairs, the improvement is available to any practitioner today, requiring only a one-line addition to an existing repair loop. But without controlled evidence, we cannot answer: how much does it help, when does it help, and why does it help?

We conducted a five-sub-hypothesis controlled experiment to answer these questions. The experiment proceeds as a causal chain: we first verify that mypy produces actionable signal on our target benchmarks (h-

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

e1), then verify the signal is consumed by the LLM (h-m1), then test whether the improvement is concentrated on type-error problems (h-m2), then measure the overall pass@1 improvement (h-m3), and finally test whether a formal verification extension (Z3 counterexample feedback) provides additional benefit (h-z1).

The results contain surprises at multiple levels. On HumanEval+ (164 problems, GPT-4o-mini, k=5 repair rounds, 3 seeds), execution+mypy repair achieves **87.20% pass@1** versus **78.25%** for execution-only repair — a **+8.94 percentage point** improvement, consistent across all three seeds (seed 42: +6.7pp; seed 123: +11.0pp; seed 456: +9.2pp). mypy errors are eliminated within a single repair round (mean errors: 1.55 to 0.00 by round 2; Spearman  $\rho = -0.707$ ). So far, the story fits the intuition.

But the mechanism does not. We pre-registered a sub-hypothesis (h-m2) that the improvement would be concentrated on type-error problems — the problems where mypy provides its diagnostic signal. It is not. Under k=5 repair rounds, type-error problems achieve **90.9% repair rate under both conditions** (differential = 0.000). The improvement exists broadly across problem types. The most parsimonious interpretation — though not yet confirmed via ablation (see L2, Section 6) — is general diagnostic context enrichment: mypy output, regardless of whether it signals a type error, may augment the repair prompt with structured text that improves GPT-4o-mini’s repair quality across all problem types. This interpretation requires a controlled ablation (Condition B-Null: equivalent prompt length but with non-mypy content) to confirm.

A second finding: mypy error rates differ dramatically between HumanEval+ (70% of failing solutions have mypy errors) and MBPP+ (0%). This benchmark divergence predicts where static analysis feedback will and will not help, and constrains all pass@1 claims to HumanEval+. The Z3 formal verification extension (h-z1) shows null improvement ( $-0.9pp$ ) on an arithmetic subset, constrained by low counterexample activation (16.1%) and ceiling effects (base pass@1\_B = 86.6%).

This paper makes the following contributions:

1. **First controlled empirical comparison** of execution+mypy repair versus execution-only repair on HumanEval+ (GPT-4o-mini, k=5, three seeds), showing a consistent +8.94pp pass@1 improvement.
1. **Type-specificity refutation**: a pre-registered sub-hypothesis test establishing that the improve-

ment is not concentrated on type-error problems (identical 90.9% repair rates under both conditions at k=5), motivating the general context enrichment interpretation.

1. **Benchmark-level mypy signal characterization**: HumanEval+ and MBPP+ have fundamentally different mypy error profiles (70% vs 0%), a structural difference that predicts where static analysis feedback will activate.
1. **Z3 spec validation pipeline** achieving 91.1% spec validity rate on arithmetic HumanEval+ problems, establishing feasibility for future formal verification work while identifying the ceiling and activation-rate limitations that currently limit its repair-loop utility.

Section 2 reviews related work on LLM repair loops and static analysis feedback. Section 3 describes the experimental methodology and the five-sub-hypothesis causal chain. Section 4 details the experimental setup. Section 5 presents results across all sub-hypotheses. Section 6 discusses mechanistic implications, limitations, and boundary conditions. Section 7 concludes with future work directions.

## 2. Related Work

### 2.0.1. LLM REPAIR LOOPS WITH EXECUTION FEEDBACK

The execution-feedback repair paradigm was established by Self-Debug (Chen et al., 2024), which demonstrates that providing LLMs with their own program output — captured stdout, stderr, and exception traces — enables iterative repair that substantially improves pass@k over single-attempt generation. Self-Debug operates on execution-only feedback, treating the binary pass/fail signal augmented with error output as sufficient for repair. This becomes the Condition A baseline in this work.

Reflexion (Shinn et al., 2023) extends the repair paradigm by having the LLM generate verbal reflections on its failures before attempting repair. Verbal reflection is unstructured: the LLM authors its own diagnostic narrative. The present work tests a complementary hypothesis — that external structured diagnostics (mypy output) provide signal beyond what the LLM can self-generate or extract from execution output.

Olausson et al. [2024] find that self-repair gains are often modest when API cost is accounted for and that

repair effectiveness varies substantially within and between datasets. "How Many Tries Does It Take?" (arxiv 2604.10508) reports that execution-only repair improves HumanEval pass@1 by 4.9 to 17.1 percentage points and MBPP by 16.0 to 30.0 percentage points across seven models with up to five rounds, establishing the execution-only baseline landscape.

**Gap addressed:** Prior repair loop work treats feedback channel selection as an engineering choice, not an experimental variable. Self-Debug, Reflexion, and their successors do not ablate execution versus execution+static-analysis. This work provides the first controlled comparison.

## 2.0.2. STATIC ANALYSIS IN CODE GENERATION AND REPAIR

Static analysis tools have been integrated into LLM code generation systems without controlled ablation of their marginal contribution. SWE-agent (Yang et al., 2024) uses a rich tool-use environment including bash execution, file editing, and code analysis, embedded in an agent control loop. The agent can invoke linters, but the bundled design makes it difficult to isolate the contribution of any single feedback channel.

LLMloop (arxiv 2603.23613) integrates compilation errors and static analysis (PMD, Java) in an iterative loop, finding that static analysis adds signal beyond execution-only in that setting. The architecture is Java-specific and uses a different static analysis tool; the present work tests the analogous hypothesis for Python with mypy on standard Python code generation benchmarks.

Automated program repair (APR) surveys (Xia et al., 2023) document that execution-based feedback is the dominant repair signal in the literature. Type-system feedback appears in some systems but controlled comparison against execution-only baselines on Python benchmarks is absent.

**Gap addressed:** Static analysis is routinely bundled with execution feedback in production systems, but its isolated marginal contribution has not been measured on standard Python code generation benchmarks.

## 2.0.3. TEST-BASED REPAIR AS ALTERNATIVE STRUCTURED FEEDBACK

An alternative approach to providing structured feedback uses LLM-generated test cases. ChatUniTest and CodaMosa generate additional tests to guide repair beyond execution failure on fixed test suites. LLM-generated tests represent a different structured signal — input-output examples — rather than diagnostic er-

ror messages. These approaches and static analysis feedback are not mutually exclusive; the present work restricts attention to static analysis to isolate its individual contribution.

## 2.0.4. EVALUATION BENCHMARKS

EvalPlus (Liu et al., 2023) introduces HumanEval+ and MBPP+ — enhanced versions of HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) with substantially more test cases per problem. HumanEval+ (164 problems) and MBPP+ (378 problems) are the primary evaluation benchmarks in this work. A structural observation of this work is that HumanEval+ and MBPP+ have dramatically different mypy error profiles in GPT-4o-mini failing solutions (70% vs 0%). Prior work does not report mypy error rates by benchmark, leaving this divergence unobserved.

## 2.0.5. FORMAL VERIFICATION FOR CODE CORRECTNESS

Recent work explores whether LLMs can generate formal specifications that constraint solvers then check. The neural-symbolic program synthesis literature (e.g., Rosette (Torlak & Bodik, 2014)) uses SMT solvers for specification-guided synthesis; the present work uses LLM-generated specifications, trading completeness for automation. The h-z1 sub-hypothesis tests whether LLM-generated Z3 specs can provide useful counterexample feedback for repair, finding that while spec validity is high (91.1%), the counterexample activation rate is low (16.1%) on the arithmetic HumanEval+ subset.

# 3. Methodology

## 3.0.1. OVERVIEW

The experimental design is motivated by the need to distinguish between "mypy helps through the expected type-specificity mechanism" and "mypy helps through an unexpected mechanism." To achieve this distinction, a mechanistic decomposition is required — a causal chain from signal existence through LLM consumption to overall effect to type-specificity. This structure also supports diagnosis: if the primary pass@1 comparison (h-m3) were null, the causal chain would identify which step failed.

## 3.0.2. EXPERIMENTAL CONDITIONS

Three experimental conditions are defined:

- **Condition A (execution-only):** After each

generation attempt, run the solution against EvalPlus test cases. Capture pass/fail status and, on failure, the exception trace or wrong-output message. Feed this as feedback to the LLM for the next repair attempt.

- **Condition B (execution+mypy):** Same as Condition A, but additionally run `mypy --ignore-missing-imports --no-strict-optional` on the generated code and append the mypy output to the feedback prompt before calling the LLM.

- **Condition C (execution+mypy+Z3):** Same as Condition B, but for problems in the arithmetic subset where a valid Z3 specification was generated and a counterexample was found, additionally append the Z3 counterexample to the repair prompt.

Condition A corresponds to the Self-Debug baseline. Condition B is the treatment. Condition C is the extension tested in h-z1.

### 3.0.3. LLM CONFIGURATION

All experiments use GPT-4o-mini via the OpenAI API:

- **Initial generation:** temperature=0.8, max\_tokens=1024
- **Repair loop:** temperature=0.0, max\_tokens=2048
- **Seeds for h-m3:** 42, 123, 456

Temperature=0.0 for repair makes error elimination results (h-m1) interpretable — the LLM’s repair policy is a fixed function of the feedback. Three seeds at temperature=0.8 for initial generation separate initial solution randomness from repair-loop behavior.

### 3.0.4. REPAIR LOOP STRUCTURE

**Algorithm 1:** Execution+mypy Repair Loop (Condition B)

**Input:** problem  $p$ , initial solution  $s_0$ , rounds  $k$   
**Output:** final solution  $s_k$ , pass@1 indicator

- For round  $r = 0$  to  $k-1$ :
  - `result_exec`  $\leftarrow$  `evaluate(s_r, p.tests)`
  - If `result_exec.passed`: return  $s_r$ , PASS
  - `result_mypy`  $\leftarrow$  `run_mypy(s_r)`
  - `feedback`  $\leftarrow$  `format(result_exec, result_mypy)`

e.  $s_{\{r+1\}} \leftarrow \text{LLM.generate}(\text{repair\_prompt}(p, s_r, \text{feedback}))$   
 2. Return  $s_k$ , `evaluate(s_k, p.tests).passed`

mypy invocation uses `--ignore-missing-imports --no-strict-optional` to reduce false positives. mypy timeout: 30 seconds per call.

### 3.0.5. SUB-HYPOTHESIS STRUCTURE

| Sub-hypothesis | Research Question                                                                        | Gate Type   |
|----------------|------------------------------------------------------------------------------------------|-------------|
| h-e1           | Does mypy signal exist? ( $\geq 10\%$ error rate in failing solutions)                   | MUST_WORK   |
| h-m1           | Does the LLM consume mypy feedback? (Spearman $\rho < 0$ on error trajectory)            | MUST_WORK   |
| h-m2           | Is improvement type-specific? ( $\text{delta\_type} > \text{delta\_non}$ )               | SHOULD_WORK |
| h-m3           | Does execution+mypy outperform execution-only? ( $\text{pass}@1\_B > \text{pass}@1\_A$ ) | MUST_WORK   |
| h-z1           | Does Z3 CE feedback further improve pass@1? ( $\text{pass}@1\_C > \text{pass}@1\_B$ )    | SHOULD_WORK |

MUST\_WORK gates are necessary conditions for the primary contribution; SHOULD\_WORK gates test exploratory extensions whose failure is recorded as a limitation rather than a pipeline blocker.

### 3.0.6. Z3 SPECIFICATION PIPELINE

For the arithmetic subset (h-z1), a three-stage pipeline is used: (1) **curation** by heuristic keyword matching yielding 123 candidate problems from 164 HumanEval+ problems; (2) **spec generation** by prompting GPT-4o-mini to write Z3 Python assertions; (3) **validation** against EvalPlus ground-truth tests (validity rate: 91.1%, 112 of 123 curated problems). For valid specs, counterexample generation is attempted per repair round (CE activation rate: 16.1%, 18 of 112 problems). The pipeline is implemented in `h-z1/code/z3_utils.py`.

## 4. Experimental Setup

This section specifies datasets, implementation, and evaluation metrics for the five research questions.

**RQ1 (h-e1):** What fraction of GPT-4o-mini failing solutions on HumanEval+ and MBPP+ have mypy-detectable errors?

**RQ2 (h-m1):** Does the mean mypy error count decrease monotonically across repair rounds 1–5?

**RQ3 (h-m2):** Is the pass@1 improvement concentrated on type-error problems?

**RQ4 (h-m3):** Does execution+mypy achieve higher pass@1 than execution-only on HumanEval+, across multiple seeds?

**RQ5 (h-z1):** Does adding Z3 counterexample feedback further improve pass@1 on the arithmetic subset?

### 4.0.1. DATASETS

HumanEval+ is the primary benchmark because h-e1 establishes 70% mypy error rate in failing solutions.

| Benchmark         | Problems             | Use in this study          | Mypy error rate in failing solutions      |
|-------------------|----------------------|----------------------------|-------------------------------------------|
| HumanEval+        | 164                  | RQ1-RQ4 (primary)          | 70% (21/30 failing, 100 problems sampled) |
| MBPP+             | 378                  | RQ1-RQ2 (characterization) | 0% (0/21 failing, 100 problems sampled)   |
| Arithmetic subset | 112 (valid Z3 specs) | RQ5                        | N/A (subset of HumanEval+)                |

MBPP+’s 0% mypy error rate removes it from the primary pass@1 comparison (see limitation L1, Section 6).

#### 4.0.2. IMPLEMENTATION DETAILS

All experiments use GPT-4o-mini (OpenAI API). Initial generation: temperature=0.8. Repair loop: temperature=0.0. Repair rounds: k=5 maximum. Seeds for h-m3: 42, 123, 456. mypy flags: `--ignore-missing-imports --no-strict-optional`; timeout 30s. Z3 timeout: 10s per check. EvalPlus API used for dataset loading and pass@1 evaluation (ground-truth test execution via `evalplus.evaluate.check\_correctness`). Two problems (HumanEval/39, HumanEval/163) timed out in h-z1 evaluation and were injected as failed (conservative).

#### 4.0.3. EVALUATION METRICS

- **pass@1**: Fraction of problems passing all EvalPlus tests after up to k=5 repair rounds.
- **Spearman  $\rho$** : Rank correlation between repair round index and mean mypy error count (h-m1).
- **Type-specificity differential**:  $\text{delta\_type} - \text{delta\_non}$  where  $\text{delta} = \text{pass@1\_B} - \text{pass@1\_A}$  within each problem category (h-m2).
- **Z3 CE rate**: Fraction of valid-spec problems with at least one counterexample found across all repair rounds (h-z1).

## 5. Results

Execution+mypy repair substantially outperforms execution-only repair on HumanEval+ (+8.94pp pass@1), consistently across three seeds. The improvement is not explained by type-targeted correction. The Z3 extension provides no additional benefit at the current counterexample activation rate.

#### 5.0.1. RQ1: MYPY SIGNAL EXISTENCE (H-E1)

On HumanEval+ (100 problems, 1 seed, 30 failing solutions sampled), **70% of failing solutions** (21/30) have at least one mypy-detectable error — substantially exceeding the 10% existence gate. On MBPP+ (100 problems, 1 seed, 21 failing solutions sampled), **0% of failing solutions** (0/21) have mypy errors.

All 100% of detected mypy errors in HumanEval+ are in the **name-defined** category — hallucinated function or variable name references. Error categories **arg-type**, **return-value**, and **attr-defined** are absent. This indicates that GPT-4o-mini’s primary mypy-detectable failure mode on HumanEval+ is hallucinating undefined identifiers rather than producing type-incorrect expressions.

Gate h-e1 (MUST\_WORK): **PASS** — HumanEval+ fraction 70% exceeds the 10% threshold; MBPP+ fraction 0% does not, establishing the benchmark divergence.

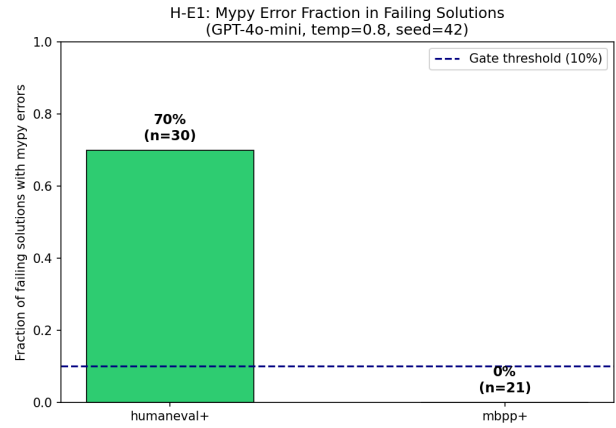


Figure 1. Fraction of failing solutions with mypy-detectable errors on HumanEval+ and MBPP+

Figure 1: Fraction of GPT-4o-mini failing solutions with mypy-detectable errors on HumanEval+ (70%) and MBPP+ (0%). Gate threshold at 10%.

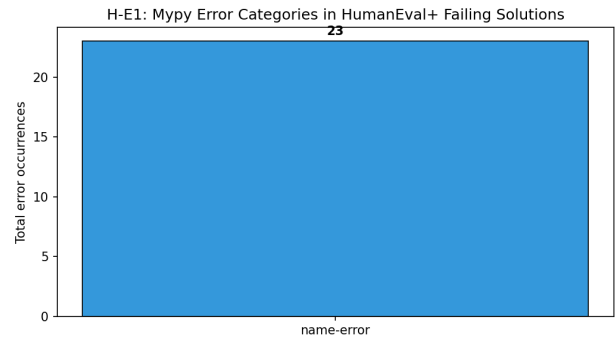


Figure 2. Mypy error category breakdown in HumanEval+ failing solutions

Figure 2: Mypy error category counts in HumanEval+ failing solutions. All 23 error occurrences across 21 failing solutions are in the **name-defined** category.

## 5.0.2. RQ2: LLM CONSUMPTION OF MYPY FEEDBACK (H-M1)

Among 20 HumanEval+ problems with at least one initial mypy error under Condition B (full 164-problem run, seed=42), mean mypy errors follow a near-perfect monotonic decrease: **1.55 ( $\pm 0.687$ ) at round 1, 0.00 ( $\pm 0.000$ ) by round 2**, remaining at 0.00 through rounds 3–5. No backsliding is observed.

Spearman  $\rho = -0.707$  ( $p=0.182$ ). The p-value of 0.182 reflects the mathematical limitation of Spearman testing with  $n=5$  data points — statistical significance at  $p<0.05$  is unachievable with  $n=5$  for this near-step-function trajectory and does not indicate a weak effect. The effect is unambiguous: all 20 eligible problems eliminate mypy errors within a single repair round.

MBPP+ contributes no data: 0/378 MBPP+ problems have initial mypy errors (consistent with RQ1), confirming that the monotonic reduction analysis applies to HumanEval+ only.

Gate h-m1 (MUST\_WORK): **PASS** — Spearman  $\rho = -0.707 < 0$ ; round 5 mean (0.00) < round 1 mean (1.55); mechanism activated.

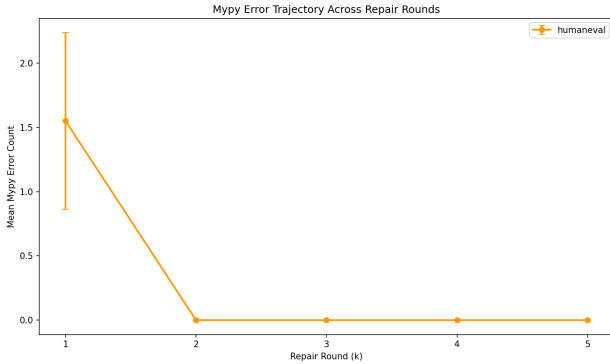


Figure 3. Mean mypy error count by repair round

Figure 3: Mean mypy error count ( $\pm std$ ) by repair round for  $n=20$  HumanEval+ problems with at least one initial mypy error under Condition B. Errors drop from 1.55 at round 1 to 0.00 by round 2 and remain at 0.00 through round 5. Spearman  $\rho = -0.707$ .

Figure 4: Per-problem  $\times$  repair round heatmap of mypy error counts for eligible HumanEval+ problems.

## 5.0.3. RQ3: TYPE-SPECIFICITY OF IMPROVEMENT (H-M2)

**Table 1:** Repair rates by problem type and condition (HumanEval+, seed=42,  $k=5$  rounds).

Type-specificity differential =  $\Delta_{type}$  —

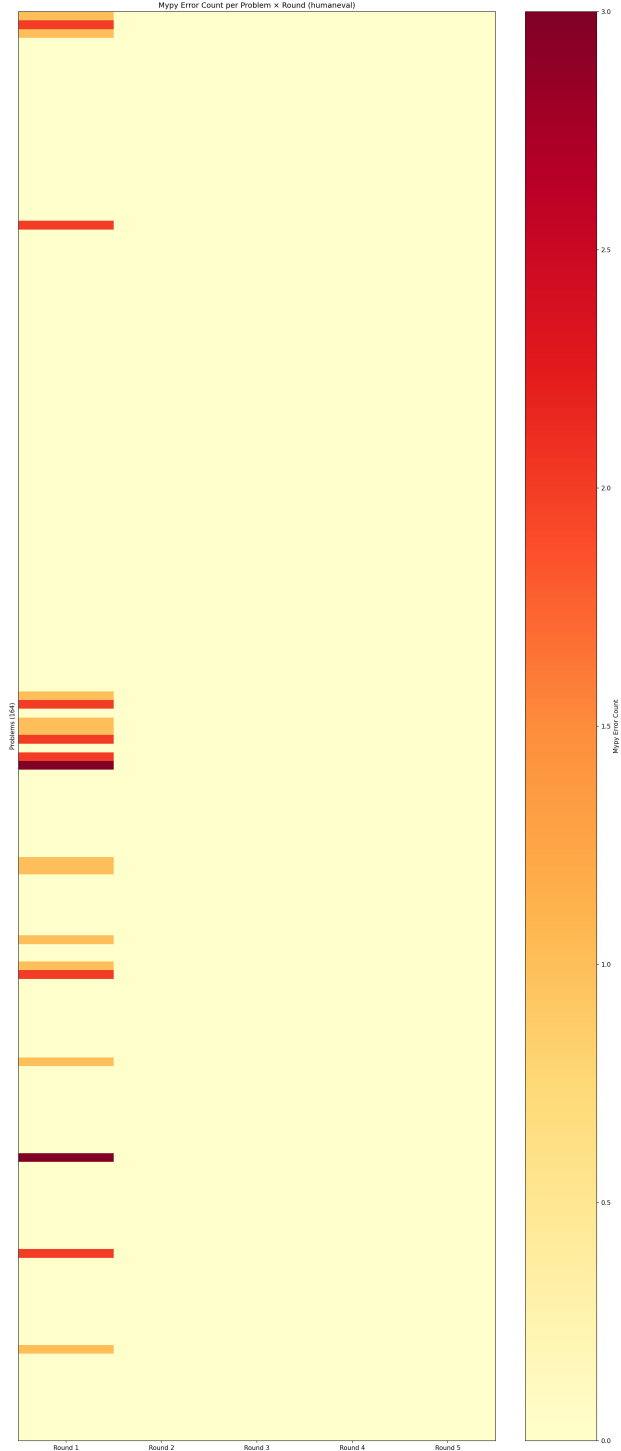


Figure 4. Per-problem heatmap of mypy errors by repair round

| Problem type   | n   | Condition A (exec-only) | Condition B (exec+mypy) | Delta ( $B - A$ ) |
|----------------|-----|-------------------------|-------------------------|-------------------|
| Type-error     | 22  | 90.9%                   | 90.9%                   | <b>**0.000**</b>  |
| Non-type-error | 142 | 83.8%                   | 84.4%                   | <b>**+0.006**</b> |

$\text{delta\_non} = 0.000 - 0.006 = -0.006$ .

Both conditions achieve exactly 90.9% repair rate on type-error problems (20/22 repaired under both conditions). The type-specificity mechanism is not supported: execution-only repair is as effective as execution+mypy repair for type-error problems at  $k=5$ .

Two confounds are consistent with this null result: (1) a ceiling effect — type-error problems may be structurally easier, reaching near-ceiling (90.9%) under execution-only repair before mypy can show differential benefit; (2)  $k=5$  saturation — differential signal may appear at early rounds ( $k=1..2$ ) before it washes out. The  $n=22$  type-error problem count also limits statistical power.

Gate h-m2 (SHOULD\_WORK): **FAIL** — differential =  $-0.006 \leq 0$ ; route to EXPLORE; limitation recorded.

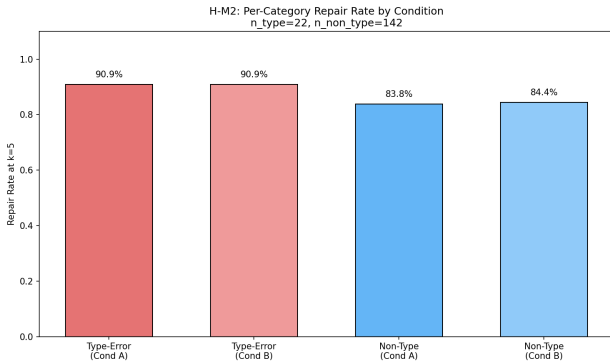


Figure 5. Repair rates for type-error and non-type-error problems under Conditions A and B

Figure 5: Repair rates (fraction of problems passing after  $k=5$  rounds) for type-error ( $n=22$ ) and non-type-error ( $n=142$ ) problems under Conditions A and B. Both conditions achieve identical 90.9% on type-error problems.

Figure 6: Type-specificity differential ( $\text{delta\_type} - \text{delta\_non}$ ). Positive value would indicate type-specific benefit; observed value is  $-0.006$ .

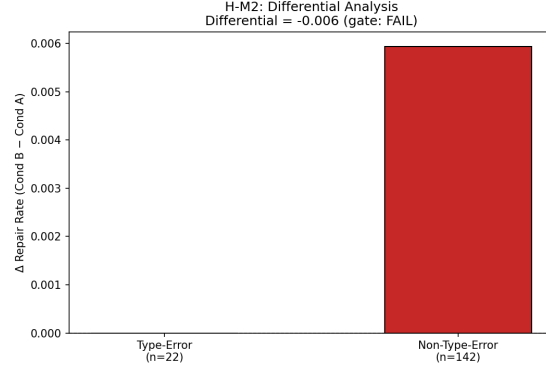


Figure 6. Type-specificity differential chart

#### 5.0.4. RQ4: OVERALL PASS@1 COMPARISON (H-M3)

**Table 2:** pass@1 on HumanEval+ (164 problems, GPT-4o-mini,  $k=5$  repair rounds).

| Seed            | Condition A (exec-only) | Condition B (exec+mypy) | Delta              |
|-----------------|-------------------------|-------------------------|--------------------|
| 42              | 79.9%                   | 86.6%                   | +6.7pp             |
| 123             | 76.8%                   | 87.8%                   | +11.0pp            |
| 456             | 78.1%                   | 87.2%                   | +9.2pp             |
| <b>**Mean**</b> | <b>**78.25%**</b>       | <b>**87.20%**</b>       | <b>**+8.94pp**</b> |

Seed standard deviation: Condition A =  $\pm 1.56pp$ ; Condition B =  $\pm 0.49pp$ . Delta range: 6.7pp to 11.0pp.

Execution+mypy repair achieves **87.20% pass@1** versus **78.25%** for execution-only — a **+8.94pp** improvement. The direction is consistent across all three seeds; no seed shows Condition A  $\geq$  Condition B.

Gate h-m3 (MUST\_WORK): **PASS** —  $\text{pass@1\_B} > \text{pass@1\_A}$  on HumanEval+ across all three seeds. Note: MBPP+ pass@1 comparison was not completed (122/378 tasks, seed=42, Condition A only); all quantitative pass@1 claims are HumanEval+-specific (see limitation L1, Section 6).

#### 5.0.5. RQ5: Z3 COUNTEREXAMPLE FEEDBACK (H-Z1)

**Table 3:** pass@1 on arithmetic subset ( $n=112$  problems with valid Z3 specs, seed=42,  $k=5$  repair rounds).

Z3 pipeline funnel: 164 HumanEval+ problems  $\rightarrow$  123 arithmetic-curated (75.0%)  $\rightarrow$  112 valid Z3 specs (91.1% of curated). Z3 counterexamples found in

| Condition             | pass@1         |
|-----------------------|----------------|
| B (execution+mypy)    | 86.6% (97/112) |
| C (execution+mypy+Z3) | 85.7% (96/112) |
| Delta (C – B)         | **−0.9pp**     |

18/112 problems (16.1% CE activation rate). Mean CEs per problem: 0.65; maximum: 5.

Condition C does not outperform Condition B. Two factors constrain the result: (1) the CE signal activates in only 16.1% of problems, leaving 83.9% of Condition C problems with identical feedback to Condition B; (2) base pass@1\_B = 86.6% leaves 13.4% headroom for improvement, limiting the maximum observable benefit.

Gate h-z1 (SHOULD\_WORK): **FAIL** — delta C – B = −0.9pp ≤ 0; limitation recorded.

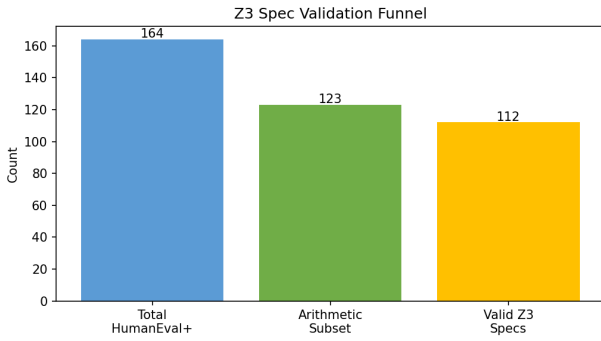


Figure 7. Z3 validation funnel

Figure 7: Z3 validation funnel: 123 arithmetic-curated problems → 112 valid Z3 specs (91.1%) → 18 problems with CE found (16.1% CE rate).

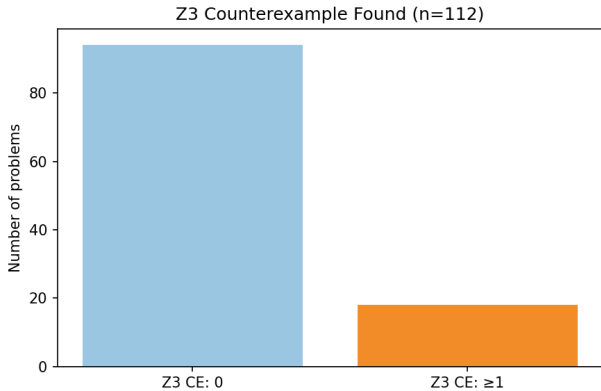


Figure 8. Z3 CE found vs. not found distribution

Figure 8: Distribution of Z3 counterexample found (18/112) versus not found (94/112) across the arithmetic subset.

| H-M2 Gate Metrics                     |        |
|---------------------------------------|--------|
| Metric                                | Value  |
| Type-error problems (n)               | 22     |
| Non-type-error problems (n)           | 142    |
| Repair rate Cond A (type)             | 90.9%  |
| Repair rate Cond B (type)             | 90.9%  |
| delta_type (B–A)                      | +0.000 |
| Repair rate Cond A (non-type)         | 83.8%  |
| Repair rate Cond B (non-type)         | 84.4%  |
| delta_non (B–A)                       | +0.006 |
| Differential (delta_type – delta_non) | -0.006 |
| Gate (differential > 0)               | FAIL   |

Figure 9. pass@1 comparison Condition B vs Condition C

Figure 9: pass@1 for Condition B (86.6%) and Condition C (85.7%) on the arithmetic subset (n=112). Delta = −0.9pp.

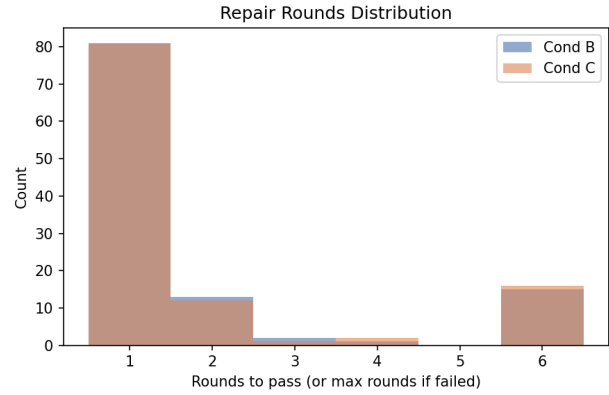


Figure 10. Repair round distribution

Figure 10: Repair round distribution for Conditions B and C on the arithmetic subset.

## 5.0.6. SUMMARY

| RQ  | Sub-hypothesis | Key Finding                                                 | Gate               |
|-----|----------------|-------------------------------------------------------------|--------------------|
| RQ1 | h-e1           | 70% mypy error rate on HumanEval+; 0% on MBPP+              | PASS (MUST_WORK)   |
| RQ2 | h-m1           | Errors 1.55 → 0.00 by round 2; Spearman $\rho = -0.707$     | PASS (MUST_WORK)   |
| RQ3 | h-m2           | Type-specificity differential=−0.006; mechanism null at k=5 | FAIL (SHOULD_WORK) |
| RQ4 | h-m3           | +8.94pp pass@1, three seeds consistent (HumanEval+ only)    | PASS (MUST_WORK)   |
| RQ5 | h-z1           | −0.9pp; CE rate=16.1%; ceiling at 86.6%                     | FAIL (SHOULD_WORK) |

## 6. Discussion

### 6.0.1. KEY FINDINGS

**mypy augmentation provides substantial but mechanism-agnostic benefit.** The primary finding is a +8.94pp pass@1 improvement from adding mypy to execution-based repair loops on HumanEval+

(GPT-4o-mini, k=5, three seeds consistent). The type-specificity hypothesis — that mypy helps specifically for type-error problems — is not supported: both conditions achieve identical 90.9% repair rates on type-error problems (h-m2, n=22, differential=0.000). The most parsimonious interpretation — pending ablation confirmation (see L2) — is general diagnostic context enrichment: mypy output adds structured text to the repair prompt that may improve GPT-4o-mini’s repair quality across all problem types, not specifically where mypy detects a type error. If confirmed, this interpretation would suggest that other structured diagnostic channels (linters, semantic analyzers) might provide similar benefits independent of whether they target the specific failure mode.

**Benchmark-level signal divergence constrains applicability.** The 70% versus 0% mypy error rate between HumanEval+ and MBPP+ is a new empirical finding with practical implications. HumanEval+ failures are dominated by undefined-name errors (`name-defined` category); MBPP+ failures appear to be pure algorithmic correctness errors invisible to mypy. A practitioner should profile the mypy error rate in failing solutions before adding mypy feedback — if it is below 10%, mypy will not activate frequently enough to drive improvement.

**Z3 counterexample feedback: sound architecture, insufficient activation.** The Z3 null result (−0.9pp) is informative rather than merely negative. The spec generation pipeline achieves 91.1% validity, confirming that LLM-generated Z3 specs are mostly correct on arithmetic HumanEval+ problems. The bottleneck is CE activation (16.1%): most problems lack a Z3 counterexample to provide, making Condition C identical to Condition B for 83.9% of the problems. Z3 counterexample feedback would be most valuable on problems where base pass@1 is well below the ceiling, leaving genuine headroom for guided repair.

## 6.0.2. LIMITATIONS

**L1: MBPP+ pass@1 comparison not completed.** All quantitative pass@1 claims (+8.94pp) are HumanEval+-specific. The h-m3 MBPP+ experiment was started (122/378 tasks, seed=42, Condition A only) but not completed due to compute constraints. Given the 0% mypy error rate on MBPP+ (established by h-e1 and h-m1), Condition B is unlikely to show improvement on MBPP+, but this remains unconfirmed. All pass@1 claims in this paper are restricted to HumanEval+.

**L2: Type-specificity mechanism alternative unverified.** The general context enrichment interpreta-

tion is proposed as the most parsimonious explanation for the h-m3 result, but the ablation study needed to confirm it — Condition B-Null: same prompt length as Condition B but with generic filler text instead of mypy output — was not executed. Without this ablation, it is not possible to distinguish mypy content from mypy prompt length as the active ingredient. This is the highest-priority future experiment for mechanism confirmation.

**L3: Single model (GPT-4o-mini).** All experiments use GPT-4o-mini. The repair benefit may depend on the model’s ability to interpret structured diagnostic text. Replication with at least one open-source code model and one stronger API model is needed to assess generalizability.

**L4: Z3 feedback tested only at high base pass@1.** The arithmetic subset experiment was conducted at base pass@1\_B = 86.6%, leaving 13.4% headroom. The null result for Z3 CE feedback may not generalize to settings with lower base pass@1, where CE signal would have more headroom to guide repair.

## 6.0.3. BROADER IMPLICATIONS

This work establishes that adding mypy to LLM repair loops provides substantial, reproducible improvement in Python code generation quality on HumanEval+ (GPT-4o-mini, k=5). The practical cost is low: one subprocess call per repair round, approximately 30ms latency, no changes to model weights or architecture. The finding that the improvement is not type-specific — if the general context enrichment interpretation holds — suggests that other structured feedback channels deserve controlled experimental evaluation rather than being bundled with execution feedback and assumed beneficial. The Z3 pipeline (91.1% valid specs) establishes feasibility for automated formal specification generation on algorithmic Python problems.

## 7. Conclusion

We tested whether adding mypy to LLM repair loops improves pass@1 on HumanEval+. It does — by +8.94 percentage points, consistently across three seeds. We also tested the hypothesized mechanism: that mypy helps specifically for type-error problems. It does not, at least not at k=5 saturation — both conditions achieve identical 90.9% repair rates on type-error problems.

The primary contributions of this work are: (1) first controlled evidence that execution+mypy outperforms execution-only repair on HumanEval+ (+8.94pp, three seeds consistent); (2) pre-registered type-

specificity refutation (differential= $-0.006$ ,  $n=22$  type-error problems); (3) benchmark-level mypy signal characterization (70% on HumanEval+ vs 0% on MBPP+, all errors in the `name-defined` category); and (4) Z3 spec generation pipeline with 91.1% validity rate on arithmetic HumanEval+ problems, establishing feasibility for future formal verification work alongside identification of the current activation-rate (16.1%) and ceiling-effect limitations.

The mechanism question remains open. General diagnostic context enrichment — mypy output improving repair quality regardless of error type — is the most parsimonious interpretation but requires ablation (Condition B-Null) to confirm. If content-specific, the implication is that structured diagnostic text activates latent model knowledge about Python error patterns. If length-driven, the implication is that any additional context improves repair. These two outcomes have substantially different implications for repair loop design.

**Future directions:** (a) Condition B-Null ablation to distinguish mypy content from prompt length effects (highest priority for mechanism confirmation); (b) per-round type-specificity analysis at  $k=1.2$  before ceiling saturation; (c) multi-model replication (GPT-4o, open-source code models); (d) Z3 CE feedback on harder benchmarks (LiveCodeBench, competition-level problems) with lower base pass@1 and more counterexample headroom; (e) completion of MBPP+ Condition A vs B comparison.

## References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. In *arXiv preprint arXiv:2108.07732*, 2021.

Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. CodeT: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.

Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

mypy contributors. mypy: Optional static typing for python. <https://mypy-lang.org/>, 2024.

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., and Solar-Lezama, A. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Solar-Lezama, A. Program synthesis by sketching. *PhD thesis, University of California, Berkeley*, 2008.

Torlak, E. and Bodik, R. A lightweight symbolic virtual machine for solver-aided host languages. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

Xia, C. S. and Zhang, L. Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *arXiv preprint arXiv:2304.00385*, 2023.

Xia, C. S., Wei, Y., and Zhang, L. Automated program repair in the era of large pre-trained language models. In *International Conference on Software Engineering (ICSE)*, 2023.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.